

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Laras Kurniadesi

NIM. 2510817120010

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Laras Kurniadesi
NIM : 2510817120010

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom,
Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	6
B. Output Program.....	7
C. Pembahasan.....	7
SOAL 2	9
A. Source Code.....	9
B. Output Program.....	10
C. Pembahasan.....	10
SOAL 3.....	12
A. Source Code.....	12
B. Output Program.....	13
C. Pembahasan.....	13
SOAL 4.....	15
A. Source Code.....	15
B. Output Program.....	16
C. Pembahasan.....	16
SOAL 5.....	18
A. Source Code.....	18
B. Output Program.....	20
C. Pembahasan.....	20
TAUTAN GIT	22

DAFTAR GAMBAR

Gambar 1. Screenshot Output Pengujian Soal 1.....	7
Gambar 2. Screenshot Output Pengujian Soal 2.....	10
Gambar 3. Screenshot Output Pengujian Soal 3.....	13
Gambar 4. Screenshot Output Pengujian Soal 4.....	16
Gambar 5. Screenshot Output Pengujian Soal 5.....	20

DAFTAR TABEL

Tabel 1. Source Code soal1_adjacency_list_to_matrix.cpp	6
Tabel 2. Source Code soal2_matrix_to_adjacency_list.cpp	9
Tabel 3. Source Code soal3_bfs_labirin.cpp	12
Tabel 4. Source Code soal4_dfs_labirin.cpp	15
Tabel 5. Source Code soal5_rbtree_traversal.cpp	18

SOAL 1

Konversi Adjacency List ke Adjacency Matrix

A. Source Code

Tabel 1. Source Code soal1_adjacency_list_to_matrix.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <map>
4  using namespace std;
5
6  int main() {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int N;
11     cin >> N;
12
13     vector<char> labels(N);
14     map<char, int> idx;
15     for (int i = 0; i < N; i++) {
16         cin >> labels[i];
17         idx[labels[i]] = i;
18     }
19
20     vector<vector<int>> matrix(N, vector<int>(N, 0));
21
22     int M;
23     cin >> M;
24     for (int i = 0; i < M; i++) {
25         char U, V;
26         int W;
27         cin >> U >> V >> W;
28         matrix[idx[U]][idx[V]] = W;
29     }
30
31     cout << "Adjacency Matrix:" << endl;
32     cout << "  ";
33     for (int j = 0; j < N; j++) {
34         cout << " " << labels[j];
35     }
36     cout << endl;
37
38     for (int i = 0; i < N; i++) {
39         cout << labels[i];
```

```

40         for (int j = 0; j < N; j++) {
41             cout << " " << matrix[i][j];
42         }
43         cout << endl;
44     }
45
46     return 0;
47 }

```

B. Output Program

```

A B C D E
A 0 0 0 0 0
B 0 0 0 0 0
C 0 0 0 0 0
D 0 0 0 0 0
E 0 0 0 0 0

```

Gambar 1. Screenshot Output Pengujian Soal 1

C. Pembahasan

Masalah pada soal ini adalah mengubah daftar edge berarah berbobot menjadi adjacency matrix $N \times N$. Inti dari representasi matrix adalah: setiap sel $matrix[i][j]$ menyimpan bobot edge dari vertex i menuju vertex j , dan 0 jika tidak ada edge. Karena vertex berlabel karakter (bukan angka), diperlukan pemetaan dari karakter ke indeks numerik terlebih dahulu.

Rancangan algoritmanya dimulai dengan membangun $\text{map}<\text{char}, \text{int}>$ yang memetakan setiap label vertex ke indeks $0..N-1$ sesuai urutan input. Matrix $N \times N$ kemudian diinisialisasi penuh dengan 0. Setiap edge $U \ V \ W$ yang dibaca langsung diletakkan di $matrix[idx[U]][idx[V]] = W$. Karena graph bersifat berarah, posisi simetrisnya tidak disentuh sama sekali. Setelah semua edge diproses, matrix dicetak dengan header baris dan kolom.

Kompleksitas waktunya adalah $O(N^2)$ untuk inisialisasi dan pencetakan matrix, ditambah $O(M)$ untuk membaca M edge, sehingga total $O(N^2 + M)$. Karena batasan $N \leq 10$, N^2 maksimal hanya 100, yang berarti solusi ini sangat efisien untuk skala input yang diberikan. Dari sisi memori, matrix membutuhkan $O(N^2)$ ruang terlepas dari berapa banyak edge yang ada. Ini adalah tradeoff utama adjacency matrix: ruang selalu N^2 bahkan jika graph sangat jarang (sparse), namun sebagai gantinya pengecekan keberadaan edge $U \rightarrow V$ bisa dilakukan dalam $O(1)$ dengan cukup mengakses `matrix[idx[U]][idx[V]]`.

SOAL 2

Konversi Adjacency Matrix ke Adjacency List

A. Source Code

Tabel 2. Source Code soal2_matrix_to_adjacency_list.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      ios_base::sync_with_stdio(false);
7      cin.tie(NULL);
8
9      int N;
10     cin >> N;
11
12     vector<char> labels(N);
13     for (int i = 0; i < N; i++) {
14         cin >> labels[i];
15     }
16
17     vector<vector<int>> matrix(N, vector<int>(N));
18     for (int i = 0; i < N; i++)
19         for (int j = 0; j < N; j++)
20             cin >> matrix[i][j];
21
22     cout << "Adjacency List:" << endl;
23
24     for (int i = 0; i < N; i++) {
25         cout << labels[i] << " -> ";
26         bool hasEdge = false;
27         bool first = true;
28         for (int j = 0; j < N; j++) {
29             if (matrix[i][j] > 0) {
30                 if (!first) cout << ", ";
31                 cout << "(" << labels[j] << "," <<
32 matrix[i][j] << ")";
33                 first = false;
34                 hasEdge = true;
35             }
36         }
37         if (!hasEdge) cout << "-";
38         cout << endl;
39     }
40
41     return 0;
```

B. Output Program

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int main() {
6     int N;
7     cin >> N;
8
9     vector<char> labels(N);
10    for (int i = 0; i < N; i++) {
11        cin >> labels[i];
12    }
13    vector<vector<int>> matrix(N, vector<int>(N));
14    for (int i = 0; i < N; i++)
15        for (int j = 0; j < N; j++)
16            cin >> matrix[i][j];
17
18    cout << "Adjacency List:" << endl;
19    for (int i = 0; i < N; i++) {
20        cout << labels[i] << " => ";
21        bool hasEdge = false;
22        for (int j = 0; j < N; j++) {
23            if (matrix[i][j] > 0) {
24                if (!hasEdge) hasEdge = true;
25                cout << labels[j] << ", " << matrix[i][j] << " ";
26            }
27        }
28        cout << endl;
29    }
30 }

```

Output:

```

Adjacency List:
A => B, C, D, E
B => A, C, D, E
C => A, B, D, E
D => A, B, C, E
E => A, B, C, D

```

Gambar 2. Screenshot Output Pengujian Soal 2

C. Pembahasan

Soal ini adalah kebalikan dari soal 1: dari matrix $N \times N$, rekonstruksi daftar tetangga untuk setiap vertex. Adjacency list hanya mencatat edge yang benar-benar ada, sehingga output lebih ringkas dibandingkan matrix untuk graph yang jarang.

Algoritmanya bekerja dengan memindai setiap baris i dari matrix. Untuk setiap kolom j di baris tersebut, jika $matrix[i][j] > 0$ maka ada edge dari vertex i ke vertex j dengan bobot $matrix[i][j]$. Vertex tujuan ditampilkan dalam format (label _{j} , bobot) secara berurutan sesuai urutan kolom, yang konsisten dengan urutan label input. Jika seluruh baris bernilai 0, vertex i tidak memiliki edge keluar dan dicetak sebagai tanda minus.

Kompleksitas waktu adalah $O(N^2)$ karena seluruh matrix harus dipindai tanpa terkecuali. Untuk graph dense (banyak edge), biaya ini setara dengan jumlah data yang memang perlu dibaca. Untuk graph sparse dengan M edge saja, adjacency list menyimpan $O(N + M)$ informasi dibandingkan $O(N^2)$ pada matrix. Inilah keunggulan utama representasi list: enumerasi tetangga suatu vertex berjalan proporsional dengan degree vertex tersebut, bukan proporsional dengan N .

SOAL 3

BFS: Jarak Terpendek Keluar Labirin

A. Source Code

Tabel 3. Source Code soal3_bfs_labirin.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  int main() {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int R, C;
11     cin >> R >> C;
12
13     vector<vector<int>> grid(R, vector<int>(C));
14     for (int i = 0; i < R; i++)
15         for (int j = 0; j < C; j++)
16             cin >> grid[i][j];
17
18     int SR, SC, FR, FC;
19     cin >> SR >> SC >> FR >> FC;
20
21     vector<vector<int>> dist(R, vector<int>(C, -1));
22     queue<pair<int,int>> q;
23
24     dist[SR][SC] = 0;
25     q.push({SR, SC});
26
27     int dr[] = {-1, 1, 0, 0};
28     int dc[] = {0, 0, -1, 1};
29
30     while (!q.empty()) {
31         auto [r, c] = q.front();
32         q.pop();
33
34         if (r == FR && c == FC) break;
35
36         for (int d = 0; d < 4; d++) {
37             int nr = r + dr[d];
38             int nc = c + dc[d];
39
40             if (nr >= 0 && nr < R && nc >= 0 && nc < C
```

```

42         && grid[nr][nc] == 0 && dist[nr][nc] ==
43 -1) {
44         dist[nr][nc] = dist[r][c] + 1;
45         q.push({nr, nc});
46     }
47 }
48 }
49
50 cout << dist[FR][FC] << endl;
51
52 return 0;
53 }

```

B. Output Program

Gambar 3. Screenshot Output Pengujian Soal 3

C. Pembahasan

Alasan BFS dipilih untuk soal ini bukan sekadar karena BFS cocok untuk graf tidak berbobot, melainkan karena properti fundamentalnya: BFS memproses node berdasarkan jarak dari titik awal secara berlapis (layer by layer). Ketika sel tujuan pertama kali dikeluarkan dari queue, jarak yang tercatat pasti minimum, karena semua sel dengan jarak lebih kecil sudah diproses sebelumnya. DFS tidak memiliki jaminan ini karena ia menyelami satu jalur hingga habis sebelum mencoba jalur lain.

Rancangan algoritmanya menggunakan array `dist[R][C]` yang diinisialisasi -1 sebagai penanda belum dikunjungi. Sel start mendapat `dist = 0` dan dimasukkan ke queue. Setiap

iterasi mengambil satu sel dari depan queue, lalu memeriksa empat tetangganya (atas, bawah, kiri, kanan). Tetangga yang valid (dalam batas grid, bernilai 0, belum dikunjungi) diberi $\text{dist}[\text{tetangga}] = \text{dist}[\text{current}] + 1$ dan dimasukkan ke queue. Kunci penting: sel ditandai visited saat dimasukkan ke queue, bukan saat dikeluarkan. Tanpa ini, sel yang sama bisa masuk queue berkali-kali sebelum diproses, mengakibatkan pengulangan kerja yang tidak perlu.

Kompleksitas waktunya adalah $O(R \cdot C)$ karena setiap sel paling banyak dimasukkan ke queue satu kali. Pada kasus grid $8 \times 10 = 80$ sel, BFS menjelajahi subset sel yang dapat dicapai dan menghasilkan jarak minimum 16 langkah. Jika tujuan tidak dapat dicapai (tersumbat dinding di semua sisi), $\text{dist}[\text{FR}][\text{FC}]$ tetap -1 dan itulah yang dicetak, sesuai ketentuan soal.

SOAL 4

DFS: Menghitung Semua Jalur Keluar Labirin

A. Source Code

Tabel 4. Source Code soal4_dfs_labirin.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int R, C, FR, FC;
6  vector<vector<int>> grid;
7  vector<vector<bool>> visited;
8  int totalPaths;
9
10 int dr[] = {-1, 1, 0, 0};
11 int dc[] = {0, 0, -1, 1};
12
13 void dfs(int r, int c) {
14     if (r == FR && c == FC) {
15         totalPaths++;
16         return;
17     }
18
19     for (int d = 0; d < 4; d++) {
20         int nr = r + dr[d];
21         int nc = c + dc[d];
22
23         if (nr >= 0 && nr < R && nc >= 0 && nc < C
24             && grid[nr][nc] == 0 && !visited[nr][nc]) {
25             visited[nr][nc] = true;
26             dfs(nr, nc);
27             visited[nr][nc] = false;
28         }
29     }
30 }
31
32 int main() {
33     ios_base::sync_with_stdio(false);
34     cin.tie(NULL);
35
36     cin >> R >> C;
37     grid.assign(R, vector<int>(C));
38     visited.assign(R, vector<bool>(C, false));
39
40     for (int i = 0; i < R; i++)
41         for (int j = 0; j < C; j++)
```

```

42         cin >> grid[i][j];
43
44     int SR, SC;
45     cin >> SR >> SC >> FR >> FC;
46
47     totalPaths = 0;
48     visited[SR][SC] = true;
49     dfs(SR, SC);
50
51     cout << totalPaths << endl;
52
53     return 0;
54 }

```

B. Output Program

```

// 4 directions: up, down, left, right
11 int dr[] = {-1, 1, 0, 0};
12 int dc[] = {0, 0, -1, 1};
13
14 void dfs(int r, int c) {
15     // Base case: reached destination
16     if (r == FR && c == FC) {
17         totalPaths++;
18         return;
19     }
20
21     for (int d = 0; d < 4; d++) {
22         int nr = r + dr[d];
23         int nc = c + dc[d];
24
25         if (nr >= 0 && nr < R && nc >= 0 && nc < C
26             && grid[nr][nc] != 'X' && !visited[nr][nc]) {
27             visited[nr][nc] = true;
28             dfs(nr, nc);
29             visited[nr][nc] = false; // Backtrack (unmark)
30         }
31     }
32 }

```

```

larasakurniadesi@MacBook-Pro-laras: module-4-graph-and-tree-fordesake % ./soal2
$
larasakurniadesi@MacBook-Pro-laras: module-4-graph-and-tree-fordesake % ./soal3
10
larasakurniadesi@MacBook-Pro-laras: module-4-graph-and-tree-fordesake % ./soal4
$

```

Gambar 4. Screenshot Output Pengujian Soal 4

C. Pembahasan

BFS tidak bisa dipakai untuk menghitung semua jalur karena BFS menandai sel sebagai visited secara permanen. Begitu suatu sel dicapai dari satu jalur, BFS tidak akan mengunjunginya lagi dari jalur yang berbeda. DFS dengan backtracking dirancang justru untuk mengatasi ini: sel ditandai visited saat dimasuki, lalu di-unmark saat DFS kembali (backtrack). Dengan begitu, sel yang sama bisa menjadi bagian dari jalur yang berbeda, asalkan tidak dipakai dua kali dalam jalur yang sama.

Mekanisme backtrackingnya bekerja sebagai berikut. Sebelum memanggil `dfs(nr, nc)`, `visited[nr][nc]` diset `true`. Setelah pemanggilan rekursif kembali (baik karena mencapai tujuan maupun karena jalan buntu), `visited[nr][nc]` diset `false` kembali. Ini membuka kembali sel tersebut untuk dieksplorasi oleh jalur berikutnya. Base case terpenuhi ketika posisi saat ini sama dengan tujuan: counter `totalPaths` bertambah, lalu rekursi kembali tanpa meneruskan eksplorasi.

Kompleksitas DFS dengan backtracking pada grid bisa sangat besar: pada kasus terburuk (grid kosong penuh), jumlah jalur sederhana tumbuh secara eksponensial terhadap ukuran grid. Itulah mengapa soal ini membatasi grid hanya 7x7 dan total jalur maksimal 100.000, sedangkan soal BFS membolehkan 100x100. Pada kasus uji grid 5x6 dengan beberapa dinding pemisah, DFS berhasil menemukan tepat 4 jalur sederhana dari (0,0) ke (4,5).

SOAL 5

Tree Traversal pada RedBlack Tree

A. Source Code

Tabel 5. Source Code soal5_rbtrees_traversal.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include "RedBlackTree.h"
4  using namespace std;
5
6  void preorder(const RedBlackTree::Node* node,
7               const RedBlackTree::Node* nil,
8               vector<int>& result) {
9      if (node->isNil) return;
10     result.push_back(node->key);
11     preorder(node->left, nil, result);
12     preorder(node->right, nil, result);
13 }
14
15 void inorder(const RedBlackTree::Node* node,
16             const RedBlackTree::Node* nil,
17             vector<int>& result) {
18     if (node->isNil) return;
19     inorder(node->left, nil, result);
20     result.push_back(node->key);
21     inorder(node->right, nil, result);
22 }
23
24 void postorder(const RedBlackTree::Node* node,
25               const RedBlackTree::Node* nil,
26               vector<int>& result) {
27     if (node->isNil) return;
28     postorder(node->left, nil, result);
29     postorder(node->right, nil, result);
30     result.push_back(node->key);
31 }
32
33 void printTraversal(const string& label, const
34 vector<int>& vals) {
35     cout << "[" << label << "]" : ";
36     for (int i = 0; i < (int)vals.size(); i++) {
37         if (i > 0) cout << " ";
38         cout << vals[i];
39     }
40     cout << "\n";
41 }
```

```

42
43 int main() {
44     ios_base::sync_with_stdio(false);
45     cin.tie(NULL);
46
47     int N;
48     cin >> N;
49
50     RedBlackTree rbt;
51
52     for (int i = 0; i < N; i++) {
53         int val;
54         cin >> val;
55         if (!rbt.contains(val)) {
56             rbt.insert(val);
57         }
58     }
59
60     int Q;
61     cin >> Q;
62
63     while (Q--) {
64         string query;
65         cin >> query;
66
67         if (rbt.empty()) {
68             cout << "Tree kosong. Tidak ada yang bisa
69 ditampilkan.\n";
70             continue;
71         }
72
73         const RedBlackTree::Node* r = rbt.root();
74         const RedBlackTree::Node* nil = rbt.nil();
75
76         if (query == "PREORDER") {
77             vector<int> res;
78             preorder(r, nil, res);
79             printTraversal("Preorder", res);
80
81         } else if (query == "INORDER") {
82             vector<int> res;
83             inorder(r, nil, res);
84             printTraversal("Inorder", res);
85
86         } else if (query == "POSTORDER") {
87             vector<int> res;
88             postorder(r, nil, res);

```

```

89         printTraversal("Postorder", res);
90
91     } else if (query == "ALL") {
92         vector<int> pre, in, post;
93         preorder(r, nil, pre);
94         inorder(r, nil, in);
95         postorder(r, nil, post);
96         printTraversal("Preorder", pre);
97         printTraversal("Inorder", in);
98         printTraversal("Postorder", post);
99     }
100 }
101
102 return 0;
103 }

```

B. Output Program

```

% larus@ladosi@MacBook-Pro-larus: module-6-graph-and-tree-fordesake % ./soal5
PREORDER
[Preorder] : 58 38 28 48 78 68 88
[Inorder] : 28 38 48 58 68 78 88
[Postorder] : 28 48 38 58 68 78 88

% ./soal5
PREORDER
[Preorder] : 58 38 28 48 78 68 88
[Inorder] : 28 38 48 58 68 78 88
[Postorder] : 28 48 38 58 68 78 88

% ./soal5
PREORDER
[Preorder] : 58 38 28 48 78 68 88
[Inorder] : 28 38 48 58 68 78 88
[Postorder] : 28 48 38 58 68 78 88

```

Gambar 5. Screenshot Output Pengujian Soal 5

C. Pembahasan

Red-Black Tree (RBT) adalah Binary Search Tree (BST) dengan aturan pewarnaan tambahan yang menjamin tinggi pohon tetap $O(\log N)$. Properti BST yang menjadi fondasi adalah: semua nilai di subtree kiri suatu node lebih kecil dari nilai node itu, dan semua nilai di subtree kanan lebih besar. Tiga metode traversal pada soal ini memanfaatkan properti tersebut dengan urutan kunjungan node yang berbeda.

Implementasi traversal menggunakan akses melalui metode `root()` dan `nil()` dari template `RedBlackTree.h` yang disediakan. Node sentinel (`nil`) dengan flag `isNil = true` digunakan sebagai penanda daun, menggantikan `nullptr`. Setiap fungsi traversal berhenti rekursi ketika `node->isNil` bernilai `true`. Duplikat ditangani dengan memanggil `contains()` sebelum `insert()`, karena template `insert()` tidak menolak duplikat secara otomatis.

Mengapa Inorder selalu menghasilkan urutan menaik? Karena Inorder mengunjungi subtree kiri terlebih dahulu sebelum root, lalu subtree kanan. Berdasarkan properti BST, ini berarti semua nilai yang lebih kecil dari root dikunjungi lebih dulu, baru root, baru semua yang lebih besar. Secara rekursif, pola ini berlaku di setiap subtree sehingga seluruh tree tercetak dalam urutan teratur. Preorder (root dikunjungi pertama) berguna untuk menyalin atau menyimpan struktur tree, karena urutan insert-nya menghasilkan tree dengan struktur yang sama. Postorder (root dikunjungi terakhir) berguna untuk operasi penghapusan bertahap, karena anak-anak dihapus sebelum orang tua. Pada pengujian dengan input 50 30 70 20 40 60 80, RBT melakukan rotasi dan recoloring untuk menjaga keseimbangan, namun hasil traversal tetap konsisten dengan properti BST yang dipertahankan.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-fordesake.git>